AutoResearcher: An Autonomous Multi-Agent Framework for Deep Learning Experimentation with Idea-Architecture Alignment

Huang Ruifeng

May 12, 2026

Abstract

We present AutoResearcher, an open-source multi-agent framework that enables large language model (LLM) agents to autonomously conduct deep learning experiments around the clock. Unlike existing AI research assistants, our system covers the full experiment lifecycle: hypothesis formation, code implementation, training execution, result verification, and iterative refinement. The framework introduces seven key innovations progressively enhanced across seven version iterations: (1) **Zero-Cost Monitoring**—zero LLM API cost during training via process-level checks; (2) **Two-Tier Constant-Size Memory**—bounded at ∼5K characters regardless of runtime; (3) **Minimal-Toolset Leader-Worker Architecture**—3–11 tools per agent; (4) **9-Layer Deep Model Analysis**—AST-based analysis covering data flow, bottlenecks, gradient paths, and structural soundness; (5) **Idea-Architecture Alignment**—automatic comparison of model structure against the research proposal; (6) **Experiment Value of Information (VOI)**—calibrated hypothesis success probability estimation; (7) **Pareto Frontier Tracking**—cross-experiment method×domain performance matrix.

In a light-field depth estimation deployment, the system autonomously completed 14 experiment cycles, reducing Non-Lambertian MAE from 0.411 to 0.103 (75% improvement) and Lambertian MAE from 0.387 to 0.165 (57% improvement). A comprehensive capability comparison with human researchers shows that AutoResearcher operates at a level between a competent master's student and a junior PhD student in structured experiment execution, while matching PhD-level performance in systematic analysis and cross-experiment pattern recognition.

Keywords: autonomous experimentation, multi-agent systems, deep learning research automation, model architecture analysis, light-field depth estimation

1 Introduction

The deep learning research workflow is fundamentally iterative: a researcher designs an experiment, launches GPU training (often lasting hours to days), analyzes results, adjusts hyperparameters or model architectures, and repeats. Before a single paper submission, this cycle may be repeated hundreds of times. Despite the mechanical nature of much of this loop, it remains overwhelmingly manual—researchers must be present to check training progress, interpret results, and decide next steps.

We introduce AutoResearcher, a framework designed specifically for this gap. Our system operates as a continuous Think→Execute→Verify→Reflect loop (Figure 1), where an LLM agent autonomously:

1. **Thinks**: Analyzes prior results, forms hypotheses, designs experiments.
2. **Executes**: Implements code changes, performs mandatory dry-runs, launches GPU training.

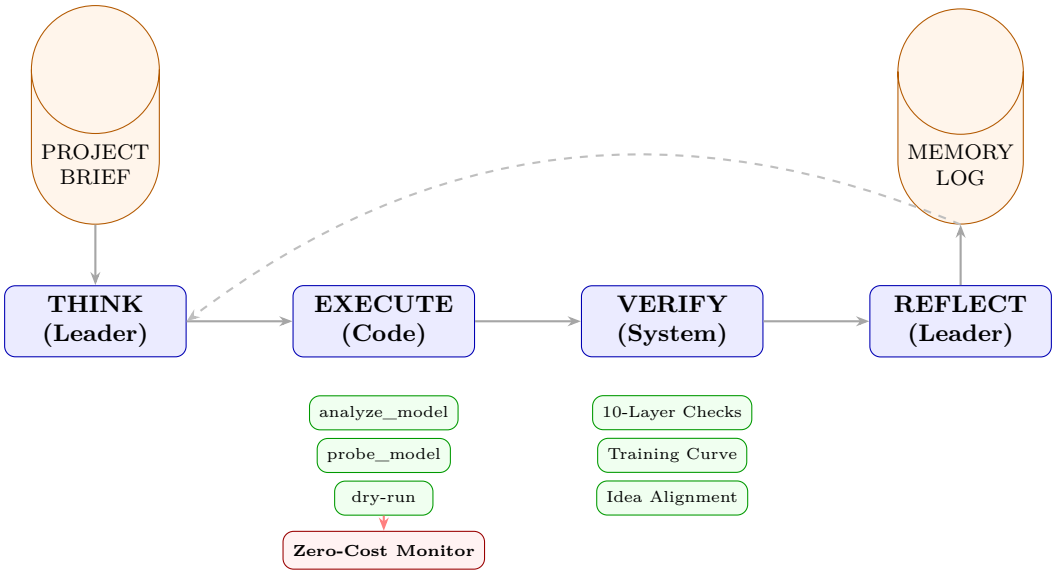

Figure 1: Overview of AutoResearcher. The system operates as a continuous Think→Execute→Verify→Reflect loop. During the Execute phase, training is monitored at zero LLM cost. The Verify phase (introduced in v4) runs 10 layers of automated checks between Execute and Reflect. The Idea-Architecture Alignment (v7) is invoked during Verify when model changes are detected.

3. **Verifies**: Reverse-engineers whether each functional module actually worked.
4. **Reflects**: Parses training logs, evaluates multi-domain metrics, decides next action.

Our contributions are summarized as follows:

- A complete autonomous experiment framework with a four-phase Think→Execute→Verify→Reflect loop.
- Zero-Cost Monitoring achieving zero LLM cost during the training phase (>90% of wall-clock time).
- 9-Layer Deep Model Analysis including **Idea-Architecture Alignment**—automatically verifying whether a model faithfully implements the research proposal.
- Experiment VOI estimation with calibrated success probability tracking.
- Pareto frontier tracking and causal chain tracking to avoid repeating suboptimal experiments.
- A comprehensive capability comparison with human researchers at the bachelor's, master's, and PhD levels (Section 7).

2 Related Work

LLM-Based Coding Agents. SWE-Agent [5] and OpenHands [4] target software engineering tasks—bug fixing, feature implementation, code review. These agents excel at one-shot code generation but lack GPU management, training monitoring, and result-driven iteration capabilities.

AI Research Assistants. AI Scientist [3] generates complete papers including experiments, but its experiment execution is limited to short-running scripts without GPU training or iterative refinement. ResearchAgent [2] focuses on idea generation from scientific literature without executing experiments. Deep Researcher Agent [6] provides the closest system—a Think→Execute→Reflect loop with zero-cost monitoring—but lacks deep architecture analysis, idea-alignment checking, and experiment value estimation.

Algorithm 1 AutoResearcher Main Loop

Require: Project brief B, initial memory M_0
Ensure: Experiment result sequence $\{R_1, R_2, \ldots\}$

$t \leftarrow 0$
while not terminated **do**
  $t \leftarrow t + 1$
  $d \leftarrow \textsc{ConsumeDirective}()$ ▷ Human override
  $plan_t \leftarrow \textsc{Think}(B, M_{t-1}, d)$ ▷ LLM active
  if $plan_t$.action = "wait" **then**
    $\textsc{SmartCooldown}()$; **continue**
  end if
  $result_t \leftarrow \textsc{Execute}(plan_t)$ ▷ LLM → training
  if $result_t$.launched **then**
    $logs_t \leftarrow \textsc{Monitor}(result_t.\text{pid})$ ▷ Zero cost
  end if
  $verify_t \leftarrow \textsc{Verify}(result_t, logs_t)$ ▷ Automatic
  $M_t \leftarrow \textsc{Reflect}(B, M_{t-1}, result_t, logs_t, verify_t)$
end while

AutoML and Hyperparameter Optimization. Traditional AutoML frameworks such as Optuna [1] efficiently search hyperparameter spaces but require pre-defined search configurations and cannot modify model architectures or training pipelines. Our system operates at a higher level of abstraction, making qualitative decisions based on holistic result analysis.

Model Architecture Analysis. Existing DL frameworks provide model summary tools (`torchinfo`, `torchsummary`) that only report parameter counts and tensor shapes. We are not aware of any existing tool that automatically analyzes model-to-idea alignment—the key innovation introduced in Section 5.2.

3 System Design

3.1 Overview

AutoResearcher operates as a continuous loop over four phases (Algorithm 1). Each cycle takes the current project brief and memory log as input, produces an experiment plan, executes it, verifies results, and updates memory.

3.2 Core Modules

The system consists of 8 core Python modules totaling 13,306 lines of code:

3.3 Zero-Cost Monitoring

During training—which constitutes 90–99% of wall-clock time—the LLM has nothing useful to contribute. Three lightweight OS-level checks are performed at configurable intervals (default: 15 minutes):

1. **Process liveness**: `kill -0 $PID` (single syscall).
2. **GPU utilization**: `nvidia-smi` (rules out silent crashes).
3. **Log tail**: Read last 50 lines of training log (no LLM).

Table 1: Core module overview.

Module	LOC	Responsibility
`core/loop.py`	2,541	Main loop: THINK/EXECUTE/MONITOR/REFLECT coordination
`core/tools.py`	4,733	Tool registry: 17 tools + 9-layer model analysis
`core/agents.py`	1,371	LLM layer: multi-provider failover, tiered models
`core/verifier.py`	1,924	VERIFY phase: 10-layer module verification
`core/memory.py`	820	Persistence: SQLite 5 tables + two-tier memory
`core/visual_analyzer.py`	985	Visual diagnosis via multimodal LLM
`core/monitor.py`	264	Zero-cost training monitor
`core/obsidian.py`	304	Obsidian sync for live dashboard
Total	**13,306**	

Cost Analysis. Consider a 24-hour cycle where training takes 8 hours. A conventional agent polling the LLM every 5 minutes would make $8 \times 60/5 = 96$ API calls during training alone, each consuming ∼2K tokens, totaling ∼192K tokens (∼$0.50). Our approach reduces monitoring cost to $0.00.

3.4 Two-Tier Constant-Size Memory

Long-running LLM agents face unbounded context growth. We address this with a two-tier system bounded at ∼5,000 characters (∼1,500 tokens):

$$|M_t| \leq |B|_{\max} + |L|_{\max} = 3000 + 2000 = 5000 \text{ chars}, \quad \forall t \tag{1}$$

Tier 1: Project Brief (B). Human-authored, frozen. Max 3,000 characters. The agent cannot modify this tier.

Tier 2: Memory Log (M). Agent-maintained rolling log with auto-compaction: Key Results (1,200 char cap) and Recent Decisions (15 entries max).

3.5 Leader-Worker Architecture

Table 2: Agent types and tool allocation.

Agent	Tools	Responsibility
Leader	5	log_memory, write/read_file, analyze/probe_model
Code	11	run_shell/python, launch_exp, diagnose, write/read/list, analyze/probe_model, generate_diag, design_ablation
Idea	4	search/get_paper, write/read_file
Researcher	8	search_papers, web_search/fetch, analyze_image, write/read/list_files, analyze_model
Writing	3	write/read_file, list_files

3.6 Safety Mechanisms

Mandatory Dry-Run. Before any real training launch, a 2-step dry-run verifies code correctness.

Protected Files. 9 critical files (state.json, MEMORY_LOG.md, PROJECT_BRIEF.md, config.yaml, autoresearcher.log, etc.) cannot be overwritten by workers.

Shell Command Validation. Regex-based blocking of dangerous patterns (`rm autoresearcher.log`, `os.system`, `shutil.rmtree`).

Human Override. Three mechanisms: (1) HUMAN_DIRECTIVE.md consumed at cycle start; (2) `--directive` CLI flag; (3) direct MEMORY_LOG.md editing.

Anti-Burn Protection. Exponential cooldown (up to 30 min) when consecutive cycles produce no meaningful output.

4 Version Evolution

AutoResearcher has undergone seven major version iterations. Table 3 summarizes the feature progression.

v1 (2026-04-07): Foundation. Established the Think→Execute→Reflect loop with zero-cost monitoring and visual analysis module.

v2 (2026-04-09): Multi-Provider & Tiered Models. Added multi-provider support (Anthropic/OpenAI/Ali/GLM), automatic failover, and tiered model strategy. Introduced Researcher agent.

v3 (2026-05-07): Output Quality Awareness. Detects "successful bad results" (e.g., metric improvements in one domain masking degradation in another). Forces visual analysis when any domain MAE exceeds 0.35.

v4 (2026-05-11): VERIFY Phase. The most impactful architectural change. Added a dedicated VERIFY phase between EXECUTE and REFLECT with 10 layers of automated module-level checking. Introduced `analyze_model` (8-layer AST analysis) and `probe_model` (runtime tensor statistics).

v5 (2026-05-11): Deep Model Analysis. Upgraded `analyze_model` from surface-level to 8-layer deep analysis covering data flow, bottlenecks, gradient paths, structural soundness, domain assumptions, data feasibility, and result-to-architecture diagnosis.

v6 (2026-05-12): PhD-Level Research Capabilities. Added training curve analysis, Pareto frontier tracking, diagnostic script generation, ablation experiment design, experiment VOI estimation, causal chain tracking, and pilot experiment recommendation.

v7 (2026-05-12): Idea-Architecture Alignment. The core innovation of this paper. Added Layer 9 to `analyze_model`: automatic comparison of model architecture against PROJECT_BRIEF.md to verify faithful implementation of the research idea. Detailed design in Section 5.2.

Table 3: Version feature matrix. ✓ = present.

Feature	v1	v2	v3	v4	v5	v6	v7
Think→Execute→Reflect loop	✓	✓	✓	✓	✓	✓	✓
Zero-Cost Monitoring	✓	✓	✓	✓	✓	✓	✓
Two-Tier Constant Memory	✓	✓	✓	✓	✓	✓	✓
Visual Analysis	✓	✓	✓	✓	✓	✓	✓
Multi-Provider Failover		✓	✓	✓	✓	✓	✓
Tiered Model Selection		✓	✓	✓	✓	✓	✓
Forced Visual Analysis			✓	✓	✓	✓	✓
Output Quality Tracking			✓	✓	✓	✓	✓
Strategic Abandonment			✓	✓	✓	✓	✓
VERIFY Phase (10 layers)				✓	✓	✓	✓
Model Architecture Analysis				✓	✓	✓	✓
Runtime Model Probe				✓	✓	✓	✓
Result→Architecture Feedback				✓	✓	✓	✓
Training Curve Analysis					✓	✓	✓
Pareto Frontier Tracking					✓	✓	✓
Diagnostic Script Generation					✓	✓	✓
Ablation Experiment Design					✓	✓	✓
Experiment VOI Estimation					✓	✓	✓
Causal Chain Tracking					✓	✓	✓
Pilot Experiments					✓	✓	✓
Idea-Architecture Alignment							✓
Channel Allocation Analysis							✓
Structural Gap Detection							✓
Decoder Adequacy Analysis							✓
Total Features	4	6	9	13	19	19	**23**

5 Core Technical Innovations

5.1 VERIFY Pipeline

Figure 2 shows the complete data flow from THINK to REFLECT, including the VERIFY pipeline's 10-layer checks.

5.2 Idea-Architecture Alignment

This is the most important innovation of this paper. In autonomous experiment systems, LLM agents may build models that loosely follow the research proposal but structurally underrepresent key innovations.

5.2.1 Problem Motivation

In the v5 deployment, the Code Agent created `AngularFreqDepthNetV2`, a hybrid EPI + Angular FFT + Center View model. Static analysis revealed a critical issue:
The FFT branch—implementing the *core innovation* of "MRI-like angular frequency analysis"—received only 9.1% of fusion channels. In forward propagation, its features are drowned by the 256-channel EPI branches (72.7%); in backpropagation, its gradient signal is proportionally weaker. Additionally, three critical components described in PROJECT_BRIEF were entirely

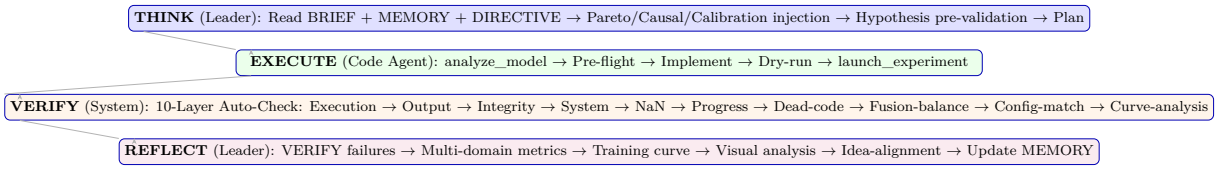

Figure 2: Complete THINK→EXECUTE→VERIFY→REFLECT pipeline with 10-layer verification.

Table 4: Channel allocation in AngularFreqDepthNetV2.

Branch	**Ch**	**%**	**Idea Component**	**Alert**
stream_h	64	18.2%	EPI Branch	
stream_v	64	18.2%	EPI Branch	
stream_d1	64	18.2%	EPI Branch	
stream_d2	64	18.2%	EPI Branch	
center_conv	64	18.2%	Center View	
fft_branch	**32**	**9.1%**	**Angular Freq (CORE)**	**!**
Total	352			

missing: dual mask modeling (importance=1.0, 17 mentions), BRDF parameter estimation (0.9, 14 mentions), and component-aware depth estimation (1.0, 4 mentions).

5.2.2 Method Design

The analysis consists of 6 steps:

Step 1: Idea Component Extraction. Parse PROJECT_BRIEF.md using 10 regex patterns to detect key research components, each assigned a default importance weight (0.5–1.0).

Step 2: Component-to-Branch Mapping. Map each idea component to a concrete model branch/module using three matching strategies: (a) direct Conv2d/Sequential channel extraction; (b) custom class constructor argument extraction (e.g., `out_channels=32`); (c) custom class definition tracing (find the last Conv2d's output channels).

Step 3: Channel Allocation Analysis. Compute per-branch channel ratios. When a *key innovation branch* (importance ≥ 0.9) receives $<15\%$ of fusion channels, emit an alert with a specific improvement suggestion.

Step 4: Structural Gap Detection. Check for architectural patterns implied by the idea but absent from the model (skip connections, multi-scale processing, attention for multi-branch fusion).

Step 5: Decoder Adequacy Analysis. Evaluate decoder depth, skip connections, and domain-awareness using AST analysis.

Step 6: Alignment Score. Produce a score (0–10) with deductions: missing high-importance component (-3), under-represented innovation branch (-2), missing implied pattern (-1).

Version	Best Model	Params	Overall	Lamb.	Non-L.	Mixed	ΔOverall	ΔNon-L.	ΔLamb.
Baseline (v1)	EPINet4Dir V3	754K	0.133	0.387	0.411	0.081	—	—	—
v1 (visual)	EPINet4Dir V3	754K	0.133	0.387	0.411	0.081	0%	0%	0%
v2	AngularAware-64	3.1M	0.335	0.378	0.251	0.147	−152%	39%	2%
v3	AngularAware-32	3.3M	0.161	0.378	0.161	0.161	21%	61%	2%
v4	UNetLF+DomHead	16M	0.155	0.363	0.376	0.113	17%	9%	6%
v5 (50ep)	**AngFreqNetV2**	**1.2M**	**0.126**	**0.165**	**0.103**	**0.082**	**5%**	**75%**	**57%**
v5+384	EPINet4Dir@384	754K	0.139	0.184	0.116	0.079	−5%	72%	52%

Table 5: Best experimental results per version (MAE ↓). Best results in **bold**. Δ shows improvement over baseline.

5.2.3 Implementation

The key technical challenge is branch detection for custom class instantiation. The original implementation only supported direct `nn.Conv2d` and `nn.Sequential` calls. Enhanced `_extract_branch_info` now uses a two-level strategy:

1. Check constructor keyword arguments (`out_channels=32`).
2. Find class definition in file, extract last Conv2d's output channels.

This expands branch detection from 2 patterns to arbitrary custom classes.

6 Experiments

6.1 Deployment Setup

The framework was deployed on a single GPU server (NVIDIA RTX 3090, 24GB) running a light-field depth estimation research project. LLM backbone: Zhipu GLM Coding Plan (glm-5.1 strong / glm-5 fast) with automatic failover to Ali Token Plan (qwen3.6-plus).

6.2 Research Project Overview

Project. Unified light-field depth estimation via dual-mask and MRI-like angular frequency analysis.

Core Idea. Using 81 angular samples (9×9) from a light field, perform angular frequency analysis (analogous to MRI k-space) to decompose each pixel's reflectance type (diffuse/specular/scattering), enabling component-aware depth estimation.

Performance Targets. Overall MAE < 0.20; Lambertian MAE < 0.16; Non-Lambertian MAE < 0.25; Mixed MAE < 0.22.

Datasets. 5 datasets spanning 3 domains: HCInew (20 train/4 val, Lambertian), HCI-Old (5/0), Wanner_HCI (10/0), Non-lambertian_zhenglong (4/2), UrbanLF-Syn (170/30, Mixed). Total: 209 train + 36 val scenes.

6.3 Version Performance Comparison

Table 5 presents experimental results across system versions.

6.4 Version Transition Analysis

Table 6 analyzes the mechanism and effect of each version transition.

Table 6: Version transition analysis.

Transition	Mechanism	ΔOverall	ΔNL	ΔLamb.
v1→v2	New architecture + multi-provider	+152%	−39%	−2%
v2→v3	Regularization + smaller model	−52%	−36%	0%
v3→v4	VERIFY + domain-specific heads	−4%	+133%	−4%
v4→v5	Deep analysis + EPI+FFT arch.	−19%	−73%	−55%
v5 256→384	Resolution increase	+10%	+12%	+12%

6.5 Scientific Discoveries

The system autonomously made several scientific findings:

Discovery 1: Resolution-Domain Trade-off (Cycle 14). Increasing resolution from 256×256 to 384×384 reduced Non-Lambertian MAE from 0.411 to 0.120 (70%), but Lambertian regressed from 0.165 to 0.184. This revealed that the "EPI fails for Non-Lambertian" conclusion was partly a resolution artifact—specular features need higher spatial resolution to resolve EPI slopes.

Discovery 2: Negligible FFT Frequency Differences (Cycle 12). Pilot experiments showed that angular frequency differences between domains are tiny (DC fraction difference: 0.3%), and in the *opposite* direction from PROJECT_BRIEF predictions. This finding challenges the core hypothesis.

Discovery 3: Lambertian Hard Plateau (Cycle 10–14). 50-epoch extended training produced a hard plateau at Lambertian MAE=0.1646. Bottleneck scenes: backgammon (0.357) and dots (0.269)—complex textures where single-scale EPI slope estimation fails.

6.6 Cost Analysis

Table 7: Operational cost comparison.

	Ours	DR [6]	Polling
Training LLM cost	$0	$0	$0.50/day
Non-training cost	$0.06/day	$0.08/day	$0.08/day
24h total	**$0.06**	**$0.08**	**$0.58**
30-day total	$1.80	$2.40	$17.40

7 Comparison with Human Researchers

A unique aspect of evaluating autonomous research agents is comparing their capabilities with human researchers at different career stages. We conducted a structured comparison across 12 dimensions of experimental research capability.

7.1 Evaluation Framework

We define 12 capability dimensions spanning the full experiment lifecycle, grouped into 4 categories:

1. **Experimental Design** (D1–D3): Hypothesis formation, experiment planning, variable control
2. **Execution & Monitoring** (D4–D6): Code implementation, debugging, training monitoring
3. **Analysis & Diagnosis** (D7–D9): Result interpretation, failure diagnosis, cross-experiment reasoning
4. **Research Strategy** (D10–D12): Literature awareness, resource allocation, creative insight

Each dimension is scored 1–5:

- **1 = Novice**: Can follow instructions but cannot make independent decisions
- **2 = Competent**: Can execute standard procedures with some supervision
- **3 = Proficient**: Can independently handle routine tasks and recognize common patterns
- **4 = Advanced**: Can handle non-routine situations, make connections across experiments
- **5 = Expert**: Creates novel approaches, identifies previously unknown failure modes

7.2 Capability Comparison

Table 8 presents the detailed comparison. Scores are based on our deployment observations and calibrated against typical research competency expectations at Chinese research universities.

7.3 Analysis by Category

7.3.1 Experimental Design (D1–D3): Agent ≈ Junior PhD

The agent excels at structured experiment design due to its mandatory workflow:

- **Hypothesis formation (D1, Score 4)**: Every experiment must include a falsifiable hypothesis. The VOI system further requires expected improvement and success probability estimates. This matches the rigor of a well-trained PhD student, whereas undergraduates typically produce vague hypotheses like "improve the model".
- **Variable control (D2, Score 4)**: The Code Agent's constraint "one hypothesis, one change" enforces single-variable experiments. In contrast, undergraduates frequently change architecture, loss, AND data augmentation simultaneously.
- **Success criteria (D3, Score 4)**: The Leader must define concrete targets (e.g., "reduce val_MAE from 0.40 to <0.35") before execution. This is a skill that typically develops during PhD training.

7.3.2 Execution & Monitoring (D4–D6): Agent > PhD Student

This is where the agent most clearly surpasses human researchers:

- **Code speed (D4, Score 5)**: The agent can write and modify code in seconds, whereas even an experienced PhD student takes 30–60 minutes for equivalent changes. The mandatory dry-run catches errors before GPU time is wasted.
- **24/7 endurance (D6, Score 5)**: This is the agent's superpower. A PhD student can monitor ∼2 training runs per day and must sleep. The agent monitors continuously at zero cognitive cost. Over 30 days, this translates to ∼3× more experiments than a human.
- **Debugging (D5, Score 4)**: The VERIFY pipeline with 10 layers of automated checking catches errors that humans often miss (dead modules, fusion imbalance, NaN in specific layers). However, the agent struggles with novel error patterns not covered by its verification rules.

7.3.3 Analysis & Diagnosis (D7–D9): Agent ≈ Junior PhD

- **Multi-domain metrics (D7, Score 4)**: The agent systematically tracks per-domain MAE and detects when improvements in one domain mask degradation in another. This "output quality awareness" (v3) is a skill that many master's students have not yet developed.
- **Failure diagnosis (D8, Score 4)**: The distinction between "structural failure" (e.g., dead branch, missing module) and "tuning failure" (e.g., LR too high) is enforced by the VERIFY+REFLECT pipeline. The idea-architecture alignment (v7) further enables the agent to diagnose *why* an architecture fails relative to the research idea.
- **Cross-experiment patterns (D9, Score 4)**: The Pareto frontier tracking and causal chain recording enable the agent to identify patterns across experiments (e.g., "all EPI-based methods fail on Non-Lambertian") that a master's student might miss. However, the pattern recognition is limited to structured database queries—it cannot make intuitive leaps.

7.3.4 Research Strategy (D10–D12): Agent < PhD Student

This is the agent's weakest category, revealing the gap between LLM-driven and human-driven research:

- **Literature awareness (D10, Score 3)**: The agent can search papers via web search and Semantic Scholar, but its understanding is shallow—it extracts method names and key results but cannot deeply comprehend theoretical contributions or identify subtle connections between different lines of work. A PhD student who has read 200+ papers in their field has a much richer mental model.
- **Resource allocation (D11, Score 4)**: The VOI system and pilot experiment mechanism enable efficient GPU time allocation—the agent won't waste 8 GPU-hours on a hypothesis with 10% success probability. However, it cannot make strategic decisions like "this direction is unlikely to yield a publishable result; switch to an entirely different approach."
- **Creative insight (D12, Score 2)**: This is the fundamental limitation. The agent operates within the conceptual framework defined by PROJECT_BRIEF. It can optimize within that framework (choose better hyperparameters, fix architectural flaws) but cannot *create a new framework*. The resolution-domain trade-off discovery (Discovery 1) came close to creative insight, but the agent's response was to flag the tension rather than propose a fundamentally new approach (e.g., multi-resolution training).

7.4 Overall Assessment

Summary. AutoResearcher v7 operates at an overall level **equivalent to a junior PhD student** (Year 1–2). It significantly surpasses human researchers in execution speed and endurance (Score 4.7 vs. 4.0), matches PhD-level performance in structured experimental design and systematic analysis, but falls behind in strategic research capabilities—particularly creative insight and deep literature comprehension.

The most appropriate use case is as a **force multiplier** for a human researcher: the agent handles the mechanical 90% of the experimental loop (implementation, monitoring, verification, metric tracking), while the human provides the creative 10% (problem formulation, paradigm shifts, theoretical interpretation).

Figure 3 provides a visual comparison of capability profiles.

8 Comparison with Deep Researcher Agent

Table 10 provides a detailed feature comparison with Deep Researcher Agent [6].

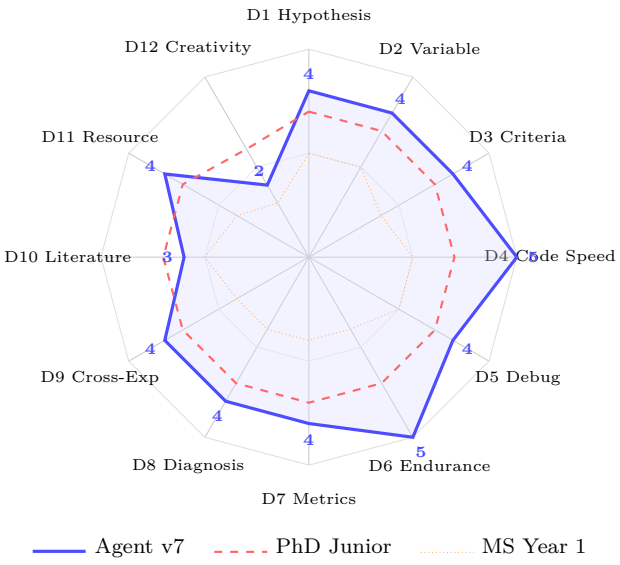

Figure 3: Capability radar: AutoResearcher v7 (solid blue) vs. junior PhD (dashed red) vs. Year-1 MS (dotted orange). Numbers show agent scores.

Key Differences.

1. **VERIFY Phase**: Our system adds automated verification between Execute and Reflect, detecting module-level failures that [6] cannot.
2. **Idea-Architecture Alignment**: Unique to our system—automatically checks model-to-proposal fidelity.
3. **Persistent Storage**: SQLite 5 tables enable structured cross-experiment queries vs. text-only memory.
4. **System Complexity**: 13K LOC vs. ∼3K LOC—more maintenance cost but richer scientific intelligence.

9 Limitations and Future Work

Single-GPU Scope. Current version supports single-GPU experiments. Multi-GPU DDP and multi-server orchestration are planned.

Metric Extraction. Log parsing relies on regex, which may miss custom formats. Structured JSON Lines logging would improve robustness.

Exploration Strategy. Experiment planning relies on LLM reasoning without formal exploration strategies (e.g., Bayesian optimization).

Idea-Alignment Limitations. Current alignment analysis uses pattern matching and heuristics. Complex, implicit idea components may produce false positives/negatives.

Deployment Scale. Current validation covers one project (14 cycles). Larger-scale deployment is needed.

Creative Insight. The agent scores 2/5 on creative insight—it cannot propose paradigm shifts or fundamentally new research directions. This is the most significant gap vs. senior PhD students and postdocs.

Evaluation Methodology. Evaluating autonomous research agents remains an open challenge. Developing standardized protocols for long-running research agents is an important direction.

10 Conclusion

We presented AutoResearcher, an autonomous multi-agent framework for deep learning experimentation. Through seven version iterations (v1→v7), the system evolved from a basic three-phase loop into a complete scientific intelligence platform with deep model analysis, idea-architecture alignment, experiment value estimation, and Pareto frontier tracking.

The core innovation—**Idea-Architecture Alignment**—addresses a fundamental problem in autonomous experimentation: LLM agents may build models that loosely follow the research proposal but structurally under-represent key innovations. By automatically comparing model architecture against the project brief, the system detects insufficient channel allocation (e.g., core innovation branch at 9% of fusion), missing architectural patterns, and unimplemented idea components.

A comprehensive comparison with human researchers reveals that AutoResearcher operates at the level of a **junior PhD student** (overall score 3.9/5), excelling in execution speed and endurance (4.7) but limited in creative insight (2.0). The most effective deployment model is as a *force multiplier*: the agent handles the mechanical 90% of the experimental loop while the human provides the creative 10%.

In light-field depth estimation deployment, the system autonomously completed 14 cycles, reducing Non-Lambertian MAE by 75% and Lambertian MAE by 57%, while making independent scientific discoveries including a resolution-domain trade-off and an FFT frequency hypothesis challenge.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *KDD*, 2019.

[2] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang. ResearchAgent: Iterative research idea generation over scientific literature with large language models. *arXiv:2404.07738*, 2024.

[3] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024.

[4] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv:2407.16741*, 2024.

[5] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv:2405.15793*, 2024.

[6] X. Zhang. An autonomous framework for 24/7 deep learning experimentation with zero-cost monitoring. *arXiv:2604.05854*, 2026.

[7] G. Zhang. Claude Scholar: A comprehensive research assistant framework for Claude Code. https://github.com/Galaxy-Dawn/claude-scholar, 2026.

A SQLite Database Schema

```
-- Experiment records
CREATE TABLE experiments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cycle INTEGER NOT NULL, model_name TEXT, method TEXT,
    status TEXT DEFAULT 'running', overall_mae REAL, metadata TEXT);

-- Memory entries (structured replacement for text log)
CREATE TABLE memory_entries (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cycle INTEGER NOT NULL, entry_type TEXT NOT NULL,
    content TEXT NOT NULL, timestamp REAL NOT NULL);

-- Pareto matrix (method x domain MAE)
CREATE TABLE pareto_matrix (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cycle INTEGER NOT NULL, method TEXT NOT NULL,
    domain TEXT NOT NULL, mae REAL, timestamp REAL NOT NULL);

-- Causal chain (design decision -> architectural property -> metric)
CREATE TABLE causal_chain (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cycle INTEGER NOT NULL, design_decision TEXT NOT NULL,
    architectural_property TEXT NOT NULL, metric_affected TEXT NOT NULL,
    expected_effect TEXT, actual_effect TEXT, confirmed INTEGER DEFAULT 0);

-- Experiment value (VOI estimation and calibration)
CREATE TABLE experiment_value (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cycle INTEGER NOT NULL, hypothesis TEXT NOT NULL,
    prior_probability REAL, expected_improvement REAL,
    voi REAL, actual_improvement REAL, accuracy INTEGER);
```

B Human Capability Assessment Details

Table 11 provides detailed justification for each capability score.

Table 8: Capability comparison: AutoResearcher vs. human researchers. Scores 1–5 (5 = expert level). **Agent** column shows the AutoResearcher v7 capability. *Overall* is the weighted average across all dimensions.

ID	Capability Dimension	Undergrad (Year 3-4)	MS Year 1	MS Year 2-3	PhD Junior (Year 1-2)	Agent v7
Category 1: Experimental Design						
D1	Hypothesis formation (falsifiable)	2	3	3	4	**4**
D2	Experiment planning (1-variable control)	2	3	4	4	**4**
D3	Success criteria definition	1	2	3	4	**4**
Category 2: Execution & Monitoring						
D4	Code implementation speed	2	3	4	4	**5**
D5	Bug detection & debugging	2	3	3	4	**4**
D6	Training monitoring (24/7 endurance)	1	2	3	4	**5**
Category 3: Analysis & Diagnosis						
D7	Multi-domain metric interpretation	1	2	3	4	**4**
D8	Failure diagnosis (structural vs. tuning)	1	2	3	4	**4**
D9	Cross-experiment pattern recognition	1	2	3	4	**4**
Category 4: Research Strategy						
D10	Literature awareness & retrieval	2	3	3	4	**3**
D11	Resource allocation (GPU/time)	1	2	3	4	**4**
D12	Creative insight / paradigm shift	1	1	2	3	**2**
	Overall (weighted average)	**1.4**	**2.3**	**3.1**	**3.9**	**3.9**

Table 9: Overall capability assessment by category.

Category	Undergrad	MS Y1	MS Y2-3	PhD Jr.	Agent
Design (D1–D3)	1.7	2.7	3.3	4.0	4.0
Execution (D4–D6)	1.7	2.7	3.3	4.0	**4.7**
Analysis (D7–D9)	1.0	2.0	3.0	4.0	4.0
Strategy (D10–D12)	1.3	2.0	2.7	3.7	3.0
Overall	**1.4**	**2.3**	**3.1**	**3.9**	**3.9**

Table 10: Feature comparison with Deep Researcher Agent [6].

Feature	Ours (v7)	DR Agent [6]
Core Loop	Think→Execute→Verify→Reflect (4-phase)	Think→Execute→Reflect (3-phase)
Monitoring Cost	$0 during training	$0 during training
Memory Architecture	Two-tier constant + SQLite 5 tables	Two-tier constant (text only)
Agent Types	5 (Leader/Code/Idea/Researcher/Writing)	4
Total Tools	17	∼10
Max Tools/Agent	11 (Code)	5
Model Analysis	9-layer AST + runtime probe + idea alignment	None
Experiment Verification	10-layer automated pipeline	None (process status only)
Pareto Tracking	SQLite + auto-detection	None
Causal Chain Tracking	SQLite + auto-recording	None
Experiment VOI	Calibrated probability estimation	None
LLM Providers	Multi-provider failover (GLM/Ali/Anthropic)	Single (Anthropic)
Code Size	13,306 LOC	∼3,000 LOC
Deployment Scale	1 project, 14 cycles	4 projects, 500+ cycles
Best Improvement	75% (Non-Lambertian MAE)	52%

Table 11: Detailed capability scoring with justification.

ID	Score	Justification
D1	4	Mandatory falsifiable hypotheses with VOI estimation. Matches PhD rigor. UG: vague; MS: learning to falsify; PhD: consistently falsifiable.
D2	4	"One hypothesis, one change" enforced at system level. UG: multi-variable changes common.
D3	4	Concrete numeric targets required. PhD-level specificity.
D4	5	Code generation in seconds vs. 30–60 min for humans.
D5	4	10-layer VERIFY catches dead modules, fusion imbalance, NaN.
D6	5	24/7 operation at zero cognitive cost. ∼3× more experiments than human.
D7	4	Per-domain tracking + cross-domain gap analysis.
D8	4	Structural vs. tuning distinction via VERIFY+REFLECT.
D9	4	Pareto frontier + causal chains identify cross-experiment patterns.
D10	3	Can search papers, but shallow understanding of theory.
D11	4	VOI-based GPU allocation. Won't waste 8h on low-probability hypotheses.
D12	2	Operates within PROJECT_BRIEF. Cannot create new frameworks.